

Biología Molecular Computacional CB309
Informe Laboratorio 01a: Needleman-Wunsch Algorithm

Alumno: Anthony Angel Briceño Quiroz

- **Introducción:**

El algoritmo de Needleman-Wunsch (1970) resuelve el problema de alineamiento global entre dos secuencias biológicas, encontrando la mejor correspondencia posible entre ellas mediante programación dinámica. Su objetivo es maximizar un score que premia coincidencias y penaliza diferencias e inserciones de gaps, permitiendo cuantificar la similitud entre secuencias de ADN, ARN o proteínas.

- **Implementación:**

La implementación se hizo en C++ y se organizó de la siguiente manera.

- **nw.h/nw.cpp:** Contiene la matriz de programación dinámica y dos versiones de traceback, una recursiva que enumera todas las soluciones óptimas y una iterativa $O(n+m)$ que reconstruye una única solución óptima, necesaria para secuencias grandes como la del archivo Secuencias.txt, donde el número de soluciones empatadas crecen exponencialmente.
- **parser.h/parser.cpp:** Lee el archivo Secuencias.txt y lo limpia.
- **dotplot.h/dotplot.cpp:** Genera la matriz de puntos.

Se aprovecho los recursos de C++: `std::vector<std::vector<int>>`, para la matriz de programación dinámica (es mejor a diferencia de arrays estáticos), se usa el paso por referencia (&) para evitar copiar matrices y strings grandes en cada llamada. `std::pair` para agrupar resultados (**nombre + secuencia, alineamiento par a par**), y recursión para el traceback exhaustivo.

- **Prueba AAAC/AGC:**

```
Matriz M:
0   -2  -4  -6
-2   1  -1  -3
-4  -1   0  -2
-6  -3  -2  -1
-8  -5  -4  -1

Score optimo: -1

Soluciones optimas encontradas: 3
Solucion 1:
AAAC
-AGC
Solucion 2:
AAAC
A-GC
Solucion 3:
AAAC
AG-C
```

- Prueba con Secuencias.txt:

```

Secuencias leídas: 3
- Bacteria: 1060 nucleotidos
- Sars-Cov: 1080 nucleotidos
- Influenza: 1080 nucleotidos

=== Alineando Bacteria vs Sars-Cov ===
Score optimo: -132
Primeros 80 caracteres de la solución optima (menos rupturas):
-TCAAGCGTTAGAGAAGTCATTATGTGATAAA--AA--AA--TTC-AACT-TGGT--ATCAACTTAACT-AAGGGTC-TT
ATTAAGGTT-TATACCTTCCCAGGTAACAAACCAACCACTTTCGATCTCTTGTAGATC-TGTTCTCTAAACGAACCTT
Dot plot guardado en: dotplot_Bacteria_vs_Sars-Cov.pgm

=== Alineando Bacteria vs Influenza ===
Score optimo: -123
Primeros 80 caracteres de la solución optima (menos rupturas):
TCAAGCGTTAGAGAAGTCATTATGTGATAAAAAATTCAACTTGGTATCAACTTAACTAA-G-GGTCTTGGTGCTG-GT-
--ATG-G--A-AGCAAT-ATCA-CTGATGACTATACTACTGGTGGTAACAAC--AAGTAATGCAGACAAAAT-CTGCATC
Dot plot guardado en: dotplot_Bacteria_vs_Influenza.pgm

=== Alineando Sars-Cov vs Influenza ===
Score optimo: -161
Primeros 80 caracteres de la solución optima (menos rupturas):
ATTAAGGTTTATACCTTCCCAGGTAACAAACCAACCACTTTCGATCTCTTGTAGATCTGTTCTCTAAACGAACCTTAA
ATGGAA-G-CAATA---TCACTGATGAC-TA--TA-CTAC--T-GGT-GGTAACA-ACAAGTAATGCAGACAAAATCT-G
Dot plot guardado en: dotplot_Sars-Cov_vs_Influenza.pgm

```

- Manejo de Cadenas Grandes – Criterio de menos rupturas:

Con secuencias de más de 1000 nucleótidos, el traceback recursivo que enumera todas las soluciones óptimas no termina en tiempo razonable, porque cada empate en la matriz genera una rama nueva y el número de soluciones crece exponencialmente. Para resolverlo se implementó tracebackUnico, que reconstruye una sola solución óptima aplicando una regla de desempate: en cada celda, si el camino diagonal (match/mismatch) es óptimo, se prioriza sobre abrir un gap nuevo; si hay que usar un gap, se prefiere continuar un bloque de gaps ya abierto en el paso anterior antes que iniciar uno nuevo. Esto minimiza el número de rupturas (bloques de gaps) en el alineamiento resultante, sin necesidad de enumerar todas las soluciones empatadas.

- Matriz de puntos:

Se implementó un dot plot con ventana deslizante (tamaño 10, umbral 7 coincidencias) para visualizar similitud entre secuencias grandes sin recalcular todo el alineamiento. Cada píxel de la imagen PGM resultante se oscurece según el número de coincidencias en esa ventana.

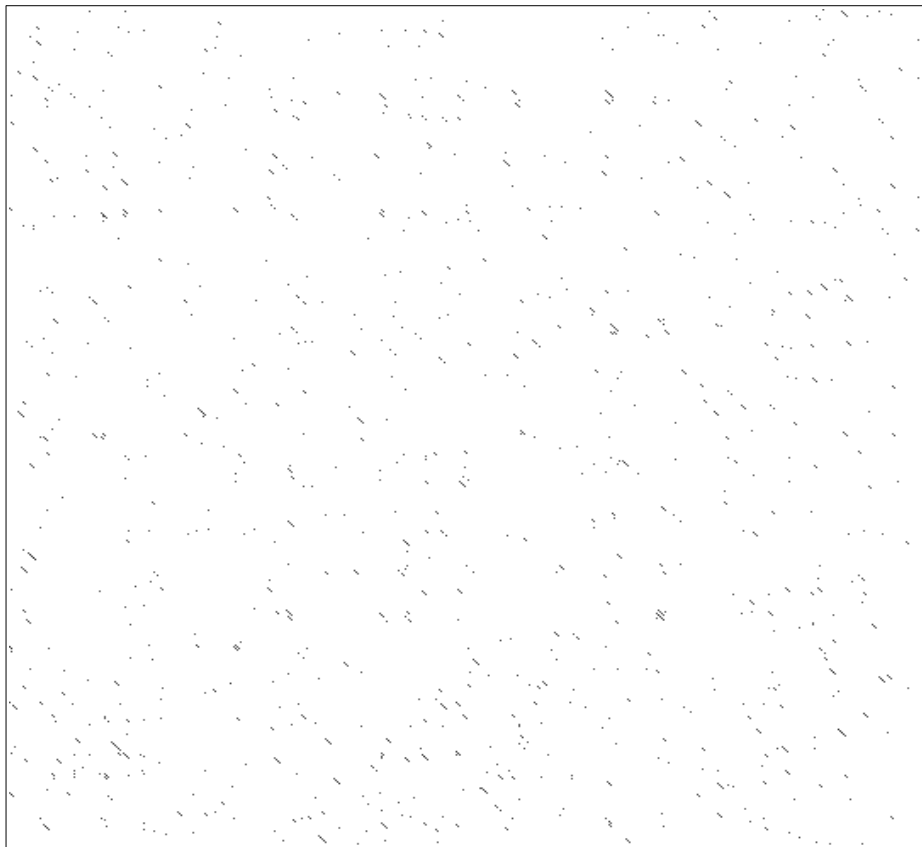


Figura 1: Dot Plot Bacteria vs Influenza



Figura 2: Dot Plot Bacteria vs Sars-Cov



Figura 3: Dot Plot Sars-Cov vs Influenza

- Preguntas:
 - Dé una revisión en la literatura buscando qué logros ha permitido alcanzar el algoritmo de Needleman-Wunsch en el área de Biología Molecular Computacional y revise también las diversas mejoras que se han investigado y logrado al mismo algoritmo. Indique cuál de las mejoras le parece la más relevante.
 - Se mejoró con el modelo de gap afín (Gotoh, 1982): en lugar de penalizar cada gap con el mismo costo fijo, se penaliza distinto el "abrir" un gap (costo alto) que el "extenderlo" (costo bajo). Esto es biológicamente más realista, porque una sola mutación de inserción/delección suele borrar o insertar varios nucleótidos seguidos de una vez, no de uno en uno
 - Si quisiera implementar este algoritmo en Python ¿Qué problemas esperaría tener y como los trataría?
 - El principal problema sería el tema del rendimiento, como tal Python es un lenguaje interpretado no compilado, por lo que llega a ser más lento que C++ y la diferencia se nota, entonces ese sería uno problema. Esto se ve reflejado en los bucles o los famosos loops, en Python si tenemos una matriz de 1K x 1K sería muy lento y tendríamos que usar librerías externas de python, mientras que en C++ se puede usar de manera rápida con pequeñas librerías integradas en el lenguaje.
 - El otro problema sería la recursión y el límite del stack, si hacemos un traceback recursivo con secuencias grandes puede exceder el límite de recursión de Python, a diferencia de C++ donde este límite es mucho más alto.
 - Las listas de listas en Python consumen más memoria por elemento que un `vector<vector<int>>` en C++, por el overhead de objetos que tiene Python. Con matrices grandes esto llegaría a ser un problema real de memoria.
- Url del Repositorio: <https://github.com/aaabriceno/CompMolecular/tree/main/labs/lab1a>